Estado Atual do Sistema — Petition Engine

Data de referência: 08 de abril de 2026
Repositório: `paulorenatolimax-ctrl/petition-engine` (GitHub, branch `main`)
Projeto local: `/Users/paulo1844/Documents/3_OMNI/_IMIGRAÇÃO/Sistema Automatizado/petition-engine/`

---

1. O QUE É O PETITION ENGINE

1.1 Contexto de Negócio (para quem não conhece imigração)

O mercado de imigração dos EUA possui categorias de visto para profissionais excepcionais que querem residir permanentemente no país (green card). As principais são:

Categoria	Para quem	Requisito central
EB-1A	Pessoas com habilidade extraordinária (artistas, cientistas, executivos)	Provar que está no topo do campo com 3+ de 10 critérios definidos pelo USCIS
EB-2 NIW	Profissionais com mestrado ou habilidade excepcional	Provar que o trabalho tem mérito e importância nacional (3 prongs do caso Dhanasar)
O-1	Similar ao EB-1A mas temporário	Habilidade extraordinária demonstrada

O processo é **administrativo** (não judicial). O peticionário submete um dossiê documental à USCIS (United States Citizenship and Immigration Services) composto por:

- **Cover Letter** — documento principal (~50-200 páginas) com toda a argumentação jurídico-estratégica
- **Résumé/CV** — perfil profissional com evidências vinculadas
- **Business Plan** — plano de negócios do empreendimento proposto (endeavor)
- **Cartas de Recomendação/Apoio** — cartas de especialistas independentes validando o perfil
- **Metodologia** — detalhamento da metodologia proprietária do peticionário
- **Declaração de Intenções** — declaração formal do empreendimento proposto

- **Evidências** — diplomas, publicações, prêmios, mídia, contratos, etc.

O Petition Engine automatiza a geração de TODOS esses documentos. O que antes levava semanas de trabalho manual por caso, o sistema gera em minutos/horas.

1.2 Visão Técnica

O Petition Engine é uma plataforma web (Next.js 14) que orquestra a geração automática de documentos de imigração usando Claude Code CLI como motor de geração. Ele:

1. **Não gera texto diretamente** — monta instruções (.md) e delega a execução ao Claude Code
2. **Não recria sistemas** — os sistemas de geração (prompts, templates, regras) já existem como arquivos .md validados empiricamente; o Petition Engine apenas os referencia via symlinks
3. **Aprende com erros** — possui um pipeline de auto-learning: erro reportado → regra criada → próxima geração já sai sem esse erro
4. **Valida antes de entregar** — quality gate local com 93 regras ativas bloqueia documentos com violações críticas
5. **Revisão cruzada** — protocolo Separation of Concerns gera revisão em sessão limpa (o agente que revisa nunca é o que escreveu)

1.3 Arquitetura de Alto Nível

```
┌──────────────────────────────────────────────────────────┐
|                     FRONTEND (Next.js 14)                 |
|  Dashboard | Clientes | Gerador | Documentos | Qualidade|
└──────────────────────┬───────────────────────────────────┘
                       | API Routes
┌──────────────────────▼───────────────────────────────────┐
|                   BACKEND (API Routes)                    |
|  /api/generate ─── Monta instrução .md                   |
|  /api/generate/execute ─── Spawna claude -p              |
|  /api/clients, /api/errors, /api/quality, etc.           |
└──────────────────────┬───────────────────────────────────┘
                       | child_process.exec
┌──────────────────────▼───────────────────────────────────┐
|              CLAUDE CODE CLI (Motor de Geração)           |
|  claude -p "Leia instrução.md e execute"                 |
|  --allowedTools Bash,Read,Write,Edit,Glob,Grep           |
|                                                           |
|  Lê: Sistema (.md) + RAGs + Dados do cliente + Regras    |
|  Gera: .docx (python-docx) ou .md (anteprojeto)         |
└──────────────────────┬───────────────────────────────────┘
                       | Arquivo gerado
┌──────────────────────▼───────────────────────────────────┐
|              QUALITY GATE (quality-local.ts)              |
|  93 regras ativas | Score 0-100 | Bloqueia se critical  |
└──────────────────────┬───────────────────────────────────┘
                       | Se aprovado
┌──────────────────────▼───────────────────────────────────┐
|          SEPARATION OF CONCERNS (Revisão Cruzada)         |
|  Sessão limpa | 4 personas | Relatório REVIEW_*.md      |
└──────────────────────────────────────────────────────────┘
```

---

2. STACK TECNOLÓGICA

Camada	Tecnologia	Versão	Função
Frontend	Next.js	14.2.33	App Router, SSR, páginas
UI	React	18	Componentes
Estilo	TailwindCSS	3.x	Dark mode, design system
Componentes	shadcn/ui	—	Componentes base (Button, Dialog, etc.)
Animações	Framer Motion	11.x	Transições de página

Camada	Tecnologia	Versão	Função
Ícones	Lucide React	—	Ícones SVG
Estado	Zustand	4.x	State management
Backend	Next.js API Routes	—	Endpoints REST
Motor de geração	Claude Code CLI	—	`claude -p` com allowedTools
Banco de dados	Supabase (PostgreSQL)	—	Clientes, documentos, regras (parcialmente migrado para JSON local)
Banco local	JSON files	—	`data/*.json` — operação offline
Scripts	Python 3.11+	—	python-docx, matplotlib, PyMuPDF
Versionamento	Git + GitHub	—	26+ commits, branch main
Node.js	—	22.20.0	Runtime

2.1 Dependências Importantes (package.json)

```
@supabase/supabase-js    - Banco de dados (usado parcialmente)
octokit                  - GitHub API (auto-commits de regras)
playwright               - Captura de screenshots (SaaS Evidence)
framer-motion            - Animações
react-hot-toast          - Notificações
zod                      - Validação de schemas
zustand                  - State management
```

2.2 Modelo de Execução

O sistema NÃO usa API paga da Anthropic para gerar documentos. Usa o **Claude Code CLI** (`claude -p`) via assinatura Pro (custo fixo mensal). Isso significa:

- Zero custo variável por documento gerado
- A geração roda localmente via `child_process.exec()`
- O Claude Code tem acesso ao filesystem (lê arquivos do sistema, evidências do cliente, RAGs)
- Output vai direto para a pasta do cliente no disco

---

3. ESTRUTURA DO PROJETO

```
petition-engine/
├── package.json
├── tsconfig.json
├── next.config.mjs
├── .env.local                          # Supabase keys, API credentials
├── .claude/                            # Documentação arquitetural (10 docs)
|   ├── 01_ARCHITECTURE.md
|   ├── 02_SUPABASE.md
|   ├── 03_AGENTS.md
|   ├── 04_API_ROUTES.md
|   ├── 05_SYSTEMS_MAP.md
|   ├── 06_ERROR_RULES.md
|   ├── 07_THUMBNAILS.md
|   ├── 08_FRONTEND.md
|   └── 09_AUTO_LEARNING.md
├── data/                               # Dados operacionais (JSON local)
|   ├── clients.json                    # 22 clientes cadastrados
|   ├── systems.json                    # 21 sistemas registrados
|   ├── error_rules.json                # 93 regras ativas
|   ├── generations.json                # 42 gerações registradas
|   ├── prompts/                        # 40+ instruções de geração
|   └── research/                       # Evidências pré-pesquisadas
├── scripts/                            # 39+ scripts Python/Node
|   ├── generate_bp_*.py                # Business Plan
|   ├── generate_resume_*.py            # Résumé
|   ├── generate_cl_*.py                # Cover Letter
|   ├── generate_saas_evidence_*.py     # SaaS Evidence
|   ├── insert_saas_screenshots.py      # Inserção de screenshots
|   ├── quality_check_*.py              # Validação
|   ├── validate_*.py                   # Validação
|   ├── generate_pptx_v2.py             # Gerador PPTX (Metodologia/Declaração)
|   └── setup-symlinks.sh               # Inicialização de symlinks
├── systems/                            # Symlinks para sistemas reais
|   ├── cover-letter-eb1a -> /...       # 24 arquivos, 244K
|   ├── resume-eb2-niw -> /...          # 4 arquivos, 46K
|   ├── business-plan -> /...           # 49 arquivos
|   └── ...                             # 21 sistemas no total
├── src/
|   ├── agents/                         # 7 agentes especializados
|   |   ├── extractor.ts                # Extrai docs do cliente
|   |   ├── writer.ts                   # Monta prompts de geração
|   |   ├── quality.ts                  # Validação via Supabase
|   |   ├── quality-local.ts            # Validação via JSON local (ATIVO)
|   |   ├── uscis-reviewer.ts           # Simula adjudicação USCIS
|   |   ├── auto-debugger.ts            # Feedback → regra automática
|   |   └── system-updater.ts           # Versionamento de sistemas
```

```
|   ├── lib/
|   |   ├── system-map.ts              # Mapa de 21 sistemas + metadata
|   |   ├── supabase.ts                # Cliente Supabase
|   |   ├── anthropic.ts               # Cliente Anthropic
|   |   ├── gemini.ts                  # Cliente Gemini (pesquisa)
|   |   └── github.ts                  # Cliente GitHub (auto-commit)
|   └── app/
|       ├── layout.tsx                  # Layout global (sidebar, dark mode)
|       ├── globals.css                 # Estilos TailwindCSS
|       ├── page.tsx                    # Dashboard
|       ├── clientes/                   # CRUD de clientes
|       ├── documentos/                 # Lista + importação + feedback
|       ├── gerador/                    # Interface principal de geração
|       |   ├── page.tsx                # Seleção de cliente + sistema
|       |   └── status/page.tsx         # Acompanhamento em tempo real
|       ├── qualidade/                  # Dashboard de qualidade
|       ├── erros/                      # Painel de error_rules
|       ├── sistemas/                   # Status dos sistemas
|       └── api/                        # 25+ endpoints REST
|           ├── generate/
|           |   ├── route.ts            # POST - monta instrução
|           |   ├── execute/route.ts    # POST - executa claude -p
(orquestrador principal)
|           |   └── complement/route.ts # POST - geração complementar
|           ├── clients/route.ts        # GET/POST clientes
|           ├── documents/route.ts      # GET/POST documentos
|           ├── errors/route.ts         # GET/POST regras de erro
|           ├── quality/route.ts        # GET stats de qualidade
|           ├── systems/route.ts        # GET sistemas
|           └── dashboard/route.ts      # GET KPIs
└── docs/
    └── handoff/                        # Esta documentação
```

---

4. OS 7 AGENTES DO SISTEMA

Cada agente é um módulo TypeScript em `src/agents/` com responsabilidade específica.

4.1 ExtractorAgent (`extractor.ts`)

Função: Lê a pasta de documentos do cliente e extrai informações relevantes (CV, diplomas, certificados, evidências).

Input: Caminho da pasta do cliente + tipo de visto
Output: Lista de arquivos extraídos com scoring de prioridade + perfil JSON do cliente

Estado: Funcional. Usado internamente pelo pipeline.

4.2 WriterAgent (`writer.ts`)

Função: Monta o prompt de geração combinando sistema (.md) + dados do cliente + RAGs + error_rules.

Input: Client ID, doc_type, system_name
Output: Prompt montado + metadata (tokens estimados, versão, regras aplicadas)

Estado: Funcional. É o coração do pipeline — toda geração passa por ele.

4.3 QualityAgent (`quality.ts`) — Supabase

Função: Validação de documentos via regras armazenadas no Supabase.

Input: Texto do documento, doc_type, nome do cliente
Output: Score, passed boolean, lista de violações

Estado: Funcional mas **parcialmente substituído** pelo quality-local.ts (que usa JSON local em vez de Supabase).

4.4 QualityAgent Local (`quality-local.ts`) — AGENTE ATIVO PRINCIPAL

Função: Validação offline de documentos contra as 93 regras em `data/error_rules.json`. Executa 5 camadas de verificação:

1. **Error Rules Check** — aplica todas as regras ativas (globais + específicas do doc_type) via regex
2. **Chain-of-Thought Detection** — detecta e remove "pensamentos em voz alta" do modelo que vazam no documento
3. **Orphan Headings Check** — detecta headings markdown (#) ou headings soltos sem conteúdo
4. **Accent Check** — verifica acentuação portuguesa (60+ palavras monitoradas)
5. **Structural Checks** — nome do cliente presente, documento não muito curto, linhas vazias excessivas
6. **Forbidden Content** — termos proibidos hardcoded (PROEX, Kortix, Carlos Avelino, prompt)
7. **BP-Specific** — para Business Plans: verifica gráficos e footnotes

Scoring:

- Cada violação critical: -25 pontos
- Cada violação high: -15 pontos
- Cada violação medium: -5 pontos
- Cada violação low: -2 pontos
- `passed = score >= 80 AND criticalCount === 0`

Comportamento no pipeline:

- Se `passed = true` → documento entregue
- Se `passed = false` E tem violações critical → **BLOQUEADO** (documento NÃO entregue, pipeline para)
- Se `passed = false` sem critical → documento entregue com ressalvas

Estado: Funcional e ativo. Recentemente corrigido para funcionar com arquivos .md (antes só funcionava com .docx).

4.5 USCISReviewerAgent (`uscis-reviewer.ts`)

Função: Simula a perspectiva de um oficial de imigração da USCIS avaliando o documento.

Input: Texto do documento, tipo de visto (EB-1A, EB-2 NIW, O-1)
Output: Avaliação de risco por critério (green/yellow/red)

Estado: Funcional mas **NÃO integrado no pipeline de execução** (`execute/route.ts`). Existe como agente standalone mas não é chamado automaticamente durante a geração.

Oportunidade de melhoria: Integrar no pipeline como Fase 1.6 (após Quality Gate, antes de SOC).

4.6 AutoDebuggerAgent (`auto-debugger.ts`)

Função: Classifica erros reportados e propõe novas regras para o `error_rules.json`.

Input: Descrição do erro, doc_type
Output: Classificação do erro (tipo, severidade, padrão regex), proposta de regra, proposta de commit no GitHub

Estado: Funcional mas **só roda em falhas técnicas** (crash, timeout). Não roda em problemas de conteúdo (que são os mais comuns). Também não é chamado automaticamente — depende de feedback manual via UI.

Oportunidade de melhoria: Integrar com o quality gate — quando o quality gate reprovar um documento, o auto-debugger deveria analisar as violações e propor regras

automaticamente.

4.7 SystemUpdaterAgent (`system-updater.ts`)

Função: Propõe atualizações nos arquivos de sistema (.md) com base em feedback acumulado.

Input: Feedback + nome do sistema
Output: Proposta de atualização (diff before/after), entrada no changelog

Estado: Funcional mas **NÃO integrado no pipeline**. Existe como agente standalone.

Oportunidade de melhoria: Conectar ao ciclo de auto-learning — após N gerações, o system-updater deveria sugerir melhorias nos prompts base.

---

5. OS 21 SISTEMAS DE GERAÇÃO

Cada "sistema" é um conjunto de arquivos .md com instruções validadas empiricamente para gerar um tipo específico de documento. O Petition Engine **não recria** esses sistemas — apenas os referencia via symlinks.

5.1 Sistemas de Geração de Documentos

#	Sistema	doc_type	Arquivos	Modelo Recomendado	Status
1	Résumé EB-2 NIW	`resume_eb2_niw`	4 (46K)	claude-sonnet-4	Funcional testado
2	Résumé EB-1A	`resume_eb1a`	9 (76K)	claude-sonnet-4	Funcional
3	Cover Letter EB-1A	`cover_letter_eb1a`	24 (244K)	claude-sonnet-4	Funcional orquestra multi-fase
4	Cover Letter EB-2 NIW	`cover_letter_eb2_niw`	18 (220K)	claude-sonnet-4	Funcional orquestra multi-fase
5	Business Plan	`business_plan`	49	claude-opus-4	Funcional testado
6	Metodologia	`methodology`	5 (65K)	claude-sonnet-4	Funcional gera PPT
7	Declaração de Intenções	`declaration_of_intentions`	6 (80K)	claude-sonnet-4	Funcional gera PPT

#	Sistema	doc_type	Arquivos	Modelo Recomendado	Status
8	IMPACTO	`impacto_report`	9 (340K)	claude-opus-4	Funcional
9	Estratégia EB-2 NIW	`strategy_eb2`	9 (64K)	claude-opus-4	Funcional
10	Estratégia EB-1A	`strategy_eb1`	3 (129K)	claude-opus-4	Funcional
11	Localização	`location_analysis`	2 (1.1M)	gemini-2.0-flash	Funcional
12	Pareceres da Qualidade	`quality_report`	1 (655K)	claude-sonnet-4	Referênci interna
13	Cartas Satélite EB-1A	`satellite_letter_eb1`	6	claude-sonnet-4	Funcional
14	Cartas Satélite EB-2 NIW	`satellite_letter_eb2`	—	claude-sonnet-4	Em construçã

5.2 Sistemas Estratégicos (Pré-Geração)

#	Sistema	doc_type	Função
15	Case Compass EB-2 NIW	`anteprojeto_eb2_niw`	Gera 3 endeavors para escolha do cliente
16	Case Compass EB-1A	`anteprojeto_eb1a`	Mapeia 10 critérios EB-1A e força de cada um
17	Case Blueprint EB-2 NIW	`projeto_base_eb2_niw`	Projeto completo após escolha do endeavor
18	Case Blueprint EB-1A	`projeto_base_eb1a`	Projeto completo com critérios priorizados

5.3 Sistemas Adicionais

#	Sistema	doc_type	Função
19	SaaS Evidence Architect	`saas_evidence`	Gera DOCX + spec Lovable para construir SaaS do cliente
20	RFE Response	`rfe_response`	Resposta a Request for Evidence do USCIS
21	Relatório Fotográfico	`photographic_report`	Report fotográfico de evidências

5.4 Fluxo de Geração (ordem correta)

```
1. Case Compass (Anteprojeto)
   └── Apresenta 3 endeavors potenciais → cliente escolhe 1
       |
2. Case Blueprint (Projeto-Base)
   └── Projeto completo com o endeavor escolhido
       |
3. Geração dos documentos do dossiê (em qualquer ordem):
   ├── Cover Letter (documento principal, ~50-200 páginas)
   ├── Résumé (perfil profissional com evidências)
   ├── Business Plan (plano de negócios)
   ├── Metodologia (método proprietário)
   ├── Declaração de Intenções (empreendimento proposto)
   ├── Cartas Satélite (recomendação/apoio, 5-10 por caso)
   └── SaaS Evidence (se aplicável)
```

---

6. PIPELINE DE GERAÇÃO (Como um documento nasce)

6.1 Fluxo Completo

```
Usuário clica "Gerar" no frontend (/gerador)
    |
    ▼
POST /api/generate
    | Monta instrução .md combinando:
    | - Sistema (arquivos .md do doc_type)
    | - Dados do cliente (pasta de documentos)
    | - RAGs (base de conhecimento jurídico-estratégico)
    | - 93 error_rules ativas (globais + específicas)
    | - Regras absolutas (segredo industrial, terminologia, etc.)
    | Salva em data/prompts/GERAR_[tipo]_[cliente].md
    |
    ▼
POST /api/generate/execute (SSE streaming)
    |
    ├── FASE 0: PRE-FLIGHT
    |   └── Verifica: claude CLI existe? Instrução existe? Pasta output existe?
    |
    ├── FASE 1: GERAÇÃO
    |   └── Executa: claude -p "Leia instrução.md e execute"
    |       └── Claude Code lê tudo, gera .docx ou .md, salva na pasta do
cliente
    |
    ├── FASE 1.5: QUALITY GATE
    |   └── Extrai texto do documento gerado
    |   └── Roda quality-local.ts (93 regras)
    |   └── Se violação critical → BLOQUEIA (não entrega)
    |   └── Se passed → continua
    |
    └── FASE 2: SEPARATION OF CONCERNS
        └── Spawna nova sessão Claude (limpa)
        └── Revisa o documento com 4 personas
        └── Gera REVIEW_[documento].md com problemas encontrados
```

6.2 Pipelines Especializados

Cover Letter EB-1A — Pipeline multi-fase (10 fases):

- Gera em 4 partes (Parte 1: critérios 1-3, Parte 2: 4-6, etc.)
- Consolidação final
- Validação pós-consolidação
- Heterogeneidade anti-ATLAS (variação de font, cor, estrutura entre documentos)

Cover Letter EB-2 NIW — Pipeline multi-fase:

- Foco nos 3 prongs de Dhanasar
- Separação por seções estratégicas

Anteprojeto/Projeto-Base — Pipeline com 9 prompts sequenciais:

- Executa prompts 1-3 (anteprojeto) ou 1-9 (projeto-base) do sistema
- Output em .md (para trabalho interno, não submissão)

PPTX (Metodologia/Declaração) — Pipeline de 2 etapas:

- Etapa 1: Claude gera JSON estruturado
- Etapa 2: Script Python (`generate_pptx_v2.py`) converte JSON → PPTX profissional

---

7. SISTEMA DE AUTO-LEARNING (Error Rules)

7.1 Conceito

O Petition Engine aprende com seus próprios erros. Cada erro identificado vira uma regra que impede sua repetição.

7.2 Ciclo de Aprendizado

```
1. Documento gerado com erro
2. Paulo identifica o erro (via revisão manual ou SOC)
3. Erro reportado via UI (/erros) ou criado diretamente
4. Nova regra adicionada ao error_rules.json
5. Próxima geração: quality gate bloqueia se a regra for violada
6. Sistema nunca comete o mesmo erro duas vezes
```

7.3 Anatomia de uma Regra

```
{
  "id": "r87",
  "rule_type": "terminology",
  "rule_description": "PROEX é CONSULTORIA. NUNCA usar terminologia
jurídica.",
  "rule_pattern": "\\b(equipe jurídica|advogado|escritório de advocacia)\\b",
  "severity": "critical",
  "rule_action": "block",
  "doc_type": null,
  "active": true,
  "source": "feedback_paulo_2026-04-08",
  "times_triggered": 0,
  "created_at": "2026-04-08T15:00:00Z"
}
```

Campo	Função
`id`	Identificador único (r1-r93)
`rule_type`	Categoria: `forbidden_term`, `content_fence`, `terminology`, `infrastructure_leak`, `content`, `formatting`, `legal`, `logic`
`rule_description`	Descrição humana da regra
`rule_pattern`	Regex para detecção automática
`severity`	`critical` (bloqueia), `high`, `medium`, `low`
`rule_action`	`block` (rejeita), `warn` (avisa), `auto_fix` (corrige automaticamente)
`doc_type`	`null` = global (todos os docs), ou específico (`resume_eb2_niw`, `business_plan`, etc.)
`active`	Se a regra está ativa
`times_triggered`	Contador de quantas vezes a regra foi ativada

7.4 Estado Atual das Regras (93 regras)

Métrica	Valor
Total de regras	93
Regras globais (todos os doc_types)	24
Regras específicas por doc_type	69
Severidade critical	61
Severidade high	26
Severidade medium/low	6

Métrica	Valor
Ação block	67
Ação warn	22
Ação auto_fix	1
Total de ativações históricas	370+

Distribuição por doc_type:

Doc Type	Regras
Global (null)	24
Business Plan	16
Satellite Letter EB-1A	11
Methodology	10
Declaration of Intentions	10
Satellite Letter EB-2 NIW	8
Resume EB-2 NIW	4
RFE Response	2
Resume EB-1A	2
PPTX	6

Categorias de regras:

Categoria	Qtd	Exemplos
`content`	47	Conteúdo obrigatório/proibido por tipo de documento
`forbidden_term`	13	"I believe", "in conclusion", "PROEX"
`formatting`	13	Margens, fontes, cores, page breaks
`terminology`	4	Linguagem administrativa vs jurídica
`content_fence`	4	Endeavors em résumés, financeiro em résumés
`infrastructure_leak`	4	RAGs, Petition Engine, Obsidian, versionamento
`legal`	4	Dhanasar em EB-1A, histórico processual
`logic`	1	Consistência interna

---

8. SEPARATION OF CONCERNS (Revisão Cruzada)

8.1 Princípio

O agente que ESCREVE nunca é o agente que REVISA. Descoberta empírica: revisão em sessão limpa encontrou 78 erros que a sessão original não viu.

8.2 Protocolo

1. **SESSÃO 1** (Claude Code) → Gera documento
2. **SESSÃO 2** (Claude Code, sessão LIMPA) → Revisa com 4 personas:
   - **USCIS Adjudication Officer** — perspectiva do oficial que vai avaliar
   - **Elite Immigration Attorney** — perspectiva de advogado de elite (WeGreened, GCEB1)
   - **Quality Auditor** — checagem de formatação, consistência, regras
   - **First-Time Reader** — leitor leigo que verifica se o texto faz sentido
3. **Output:** `REVIEW_[documento].md` — relatório com problemas encontrados, categorizados por severidade (BLOQUEANTE, CRÍTICO, ALTO, MÉDIO), com score 0-100

8.3 Estado

Funcional e integrado no pipeline (Fase 2). Os resultados do REVIEW são salvos na pasta do cliente mas **não são gravados no generations.json** (oportunidade de melhoria).

---

9. FRONTEND — PÁGINAS

9.1 Dashboard (`/`)

Visão geral com KPIs: documentos gerados, clientes ativos, score médio de qualidade, regras ativas.

9.2 Clientes (`/clientes`)

CRUD de clientes: nome, tipo de visto, empresa, pasta de documentos. 22 clientes cadastrados atualmente.

9.3 Gerador (`/gerador`) — Página Principal

Interface de geração: seleciona cliente, seleciona sistema (doc_type), adiciona instruções específicas, clica "Gerar". O sistema monta a instrução, executa, e mostra progresso em

tempo real via SSE.

9.4 Status de Geração (`/gerador/status`)

Acompanhamento em tempo real: fases do pipeline, stages, quality gate resultado, SOC resultado.

9.5 Documentos (`/documentos`)

Lista de documentos gerados com download, importação de documentos existentes, e sistema de feedback (cirúrgico = apontamento específico, cascalho = refazer inteiro).

9.6 Qualidade (`/qualidade`)

Dashboard de qualidade: score médio, violações mais comuns, distribuição por doc_type.

9.7 Erros (`/erros`)

Painel de error_rules: visualizar regras ativas, ativar/desativar, ver contadores de ativação.

9.8 Sistemas (`/sistemas`)

Status dos 21 sistemas: nome, versão, quantidade de arquivos, modelo recomendado, se está ativo.

---

10. DADOS DE PERFORMANCE

10.1 Estatísticas de Geração (42 gerações)

Métrica	Valor
Total de gerações	42
Concluídas com sucesso	24 (57%)
Falhas	17 (40%)
Em processamento	1 (2%)
Duração média (sucesso)	~25 minutos
Doc type mais gerado	projeto_base_eb2_niw (10)

10.2 Falhas Comuns

Tipo de falha	Qtd	Causa raiz
"Exit 0 mas nenhum documento criado"	8	Instrução genérica demais ou Claude não entendeu o que gerar
"Geracao falhou (exit 1)"	4	Erro no Claude Code CLI
Timeout (30min+)	3	Documento muito grande ou servidor fechado
"Exit code 143"	2	Processo killed externamente

10.3 Oportunidades de Melhoria Identificadas

#	Problema	Impacto	Complexidade
1	USCIS Reviewer não integrado no pipeline	Alto — documentos saem sem avaliação de risco USCIS	Média
2	Auto-Debugger só roda em falhas técnicas	Alto — erros de conteúdo não geram regras automáticas	Média
3	System Updater não integrado	Médio — prompts base não melhoram automaticamente	Média
4	`stages` não eram gravados no generations.json	Baixo — corrigido em 08/abr/2026	Feito
5	Quality gate não funcionava com .md	Alto — anteprojetos/projetos-base passavam sem validação	Feito
6	Quality gate não bloqueava	Alto — documentos reprovados eram entregues igual	Feito
7	Résumés contaminados com endeavors	Alto — sem cerca de dados no prompt	Feito
8	Taxa de sucesso de 57%	Alto — 43% das gerações falham	Alta
9	Paths de clientes desatualizados	Médio — pastas renomeadas, docs_folder_path incorreto	Baixa
10	Falta de geração paralela de cartas satélite	Médio — uma carta por vez, deveria ser paralelo	Alta
11	Falta importação automática de perfil	Médio — Google Scholar, LinkedIn não integrados	Alta
12	Falta compliance com USCIS Evidence Classifier	Alto — USCIS usa IA para categorizar evidências	Alta
13	Falta sistema de RFE Response automático	Alto — demanda crescente, parcialmente construído	Alta

11. RAGs (Retrieval-Augmented Generation)

RAGs são documentos de referência jurídico-estratégica que o Claude lê ANTES de gerar qualquer documento. São a "memória institucional" do sistema — contêm análises de casos aprovados/negados, padrões de adjudicação, práticas de escritórios de elite, etc.

Localização base: `~/Documents/_PROEX (A COMPLEMENTAR)/_(RAGs) - ARGUMENTAÇÃO (ESTUDO)_LINKS QUE REFORÇAM/`

Visa	Docs	Conteúdo principal
EB-1A	4 documentos	Análise de critérios, expectativas de oficiais, práticas de escritórios de elite, pesquisa competitiva
EB-2 NIW	11 documentos	Análise de adjudicação Dhanasar, O Adjudicador Algorítmico (como IA do USCIS avalia), Construindo o Caso EB-2 NIW
O-1	Em construção	—
Geral	Vários	Jurisprudência, estratégias, padrões

Importância: Os RAGs são o diferencial competitivo principal — treinados em casos reais próprios (não dados públicos como os concorrentes usam).

---

12. REGRAS DE NEGÓCIO INVIOLÁVEIS

Estas regras são inegociáveis e devem ser respeitadas em qualquer desenvolvimento futuro:

#	Regra	Razão
1	Motor de geração é `claude -p` via CLI, NÃO API paga	Custo fixo mensal, zero custo variável
2	`--allowedTools Bash,Read,Write,Edit,Glob,Grep` obrigatório	Sem isso, Claude só cospe texto no stdout
3	Output SEMPRE .docx para documentos de submissão	USCIS não aceita .md
4	Output .md para anteprojeto/projeto-base	Documentos internos de trabalho
5	NUNCA expor infraestrutura interna (RAGs, Petition Engine, Obsidian)	Segredo industrial
6	NUNCA usar terminologia jurídica/advocatícia	PROEX é consultoria, USCIS é processo administrativo

#	Regra	Razão
7	NUNCA mencionar PROEX, Kortix, nomes de outros clientes	Confidencialidade
8	Naming: V prefix na frente (V2_Arquivo.md), NUNCA no final	Convenção do ecossistema
9	NUNCA timestamps no nome de arquivo	Usar V1, *V2*, V3_
10	Instruções estratégicas, NUNCA pseudo-código rígido	O Claude gera melhor com direção estratégica
11	Prompts >40K tokens devem ser divididos em fases	Acima disso, Claude faz meta-análise em vez de gerar
12	Heterogeneidade anti-ATLAS nas cartas satélite	Variação de font, cor, estrutura entre cartas do mesmo caso
13	Separation of Concerns obrigatório	Quem escreve nunca revisa
14	Tradução certificada (NUNCA juramentada)	Conceito brasileiro que não existe no sistema americano

---

13. CONCEITOS-CHAVE DO DOMÍNIO

Termo	Significado
Endeavor	O empreendimento/atividade proposta pelo peticionário nos EUA
Prongs (Dhanasar)	3 requisitos para EB-2 NIW: (1) mérito + importância nacional, (2) bem posicionado para avançar, (3) balanço de fatores favorece o waiver
Kazarian	Análise em 2 passos para EB-1A: Step 1 (evidências por critério) → Step 2 (totalidade das evidências)
SOC Code	Código de ocupação do Bureau of Labor Statistics — classifica a profissão do peticionário
RFE	Request for Evidence — USCIS pede mais evidências antes de decidir
ATLAS/ATA	Sistema de detecção de templates do USCIS — por isso cartas precisam ser heterogêneas
Anteprojeto	Fase inicial: apresenta 3 endeavors para o cliente escolher
Projeto-Base	Fase completa: projeto estratégico após escolha do endeavor
Cartas Satélite	Cartas de recomendação/apoio de terceiros independentes
Separation of Concerns	Protocolo de revisão cruzada em sessão limpa
Cirúrgico	Feedback específico (seção X, página Y, este parágrafo)
Cascalho	Feedback de rejeição total de um bloco/seção

Termo	Significado
Anti-Cristine	Regras que impedem termos que provam que o endeavor funciona SEM o peticionário (mata Prong 3)
Auto-peticionário	Pessoa que faz sua própria petição sem advogado (essência do EB-2 NIW)
Pareceres da Qualidade	50+ regras extraídas de revisão crítica de casos reais — inteligência qualitativa proprietária

---

14. COMO SUBIR O AMBIENTE

```shell
# 1. Clonar o repositório
git clone https://github.com/paulorenatolimax-ctrl/petition-engine.git
cd petition-engine

# 2. Instalar dependências
npm install

# 3. Configurar ambiente
cp .env.local.example .env.local
# Preencher: NEXT_PUBLIC_SUPABASE_URL, NEXT_PUBLIC_SUPABASE_ANON_KEY

# 4. Subir o servidor
npm run dev
# → http://localhost:3000

# 5. Verificar APIs
curl -s localhost:3000/api/dashboard | python3 -m json.tool
curl -s localhost:3000/api/systems | python3 -m json.tool
curl -s localhost:3000/api/errors | python3 -m json.tool

# 6. Ver regras ativas
cat data/error_rules.json | python3 -c "
import sys,json
rules = json.load(sys.stdin)
for r in rules:
    print(f'{r[\"id\"]}: [{r[\"severity\"]}] {r[\"rule_description\"][:80]}')
"

# 7. Ver clientes
cat data/clients.json | python3 -c "
import sys,json
for c in json.load(sys.stdin):
    print(f'{c[\"id\"]}: {c[\"name\"]} ({c[\"visa_type\"]})')
"
```

---

15. DOCUMENTAÇÃO ARQUITETURAL

Os seguintes documentos em `.claude/` contêm especificações detalhadas. Leia na ordem:

Arquivo	Conteúdo
`01_ARCHITECTURE.md`	Stack, princípios, diagrama do orquestrador
`02_SUPABASE.md`	Schema completo (clients, documents, error_rules, system_versions)
`03_AGENTS.md`	Especificação dos 7 agentes
`04_API_ROUTES.md`	Todos os endpoints REST
`05_SYSTEMS_MAP.md`	Symlinks + pipeline de 9 prompts do anteprojeto
`06_ERROR_RULES.md`	50 regras seed + categorias
`07_THUMBNAILS.md`	Geração de thumbnails em DOCX
`08_FRONTEND.md`	Páginas e componentes
`09_AUTO_LEARNING.md`	Sistema de auto-aprendizado (3 níveis)

---

16. AUDITORIAS REALIZADAS

Auditoria	Data	Escopo	Localização
Mega Audit	mar/2026	Auditoria técnica exaustiva de todo o sistema	`C.P./MEGA_AUDIT_PETITION_ENGINE.md`
Auditoria Sistemas 15-17	mar/2026	SOC, PPTX Engineering, Design System	`C.P./AUDIT_SISTEMAS_15_16_17.md`
Diagnóstico Fase 4	25/mar/2026	Fixes de infraestrutura (API, duplicatas, paths)	`C.P./DIAGNOSTICO_FASE4_FIXES_2026-03-25.md`
Sistema Anteprojeto	25/mar/2026	EB-2 NIW e EB-1A, 16 novas regras	`C.P./ANTEPROJETO_SISTEMA_2026-03-25.md`
Résumé V7 Fixes	25/mar/2026	Evolução do sistema de résumé	`C.P./RESUME_V7_FIXES_2026-03-25.md`
Anti-FOMO	06/abr/2026	Diagnóstico competitivo vs mercado (8 concorrentes)	`C.P./RELATORIO_ANTI_FOMO_2026-04-06.md`
Engenharia Reversa	abr/2026	Reverse-engineering de 3 benchmarks + cartas Mariana Kasza	`C.P./ENGENHARIA_REVERSA_*.md`

Auditoria	Data	Escopo	Localização
Deep Research BP	abr/2026	Pesquisa profunda sobre automação de Business Plans	`C.P./DEEP_RESEARCH_BP_*.md`

---

17. HISTÓRICO DE COMMITS RECENTES

```
2cdce87 fix: Supabase keepalive — prevent automatic pausing
d215c2f fix: correct BP system path — was empty, actually has 49 files
d203b46 fix: RFE EB-2 NIW orchestrator updated with Deni Rubens benchmark
8552e36 feat: ALL 4 orchestrators complete + fix EB-2 NIW CL path
c0f57ed feat: RFE EB-1A Orchestrator — reverse-engineered from Marcelo Góis VF
1d7188c feat: implement multi-phase Cover Letter EB-1A orchestrator
a3506f1 feat: Orchestrator V2 — validated by Cowork against Vitória VF
7e43ad0 feat: Cover Letter EB-1A Multi-Phase Orchestrator
4e029d9 fix: anti-Cristine V2 — hard blocks + case history ban + expanded terms
5b65dc0 feat: Premium Design System V2 (Cristine Style) + anti-Cristine rules
90203f8 fix: anti-Cristine corrections — nexo beneficiário↔endeavor + Prong 3
4fe3c38 docs: AUDITORIA EXAUSTIVA — 1.215 linhas, 90KB, 100+ arquivos lidos
363c863 docs: AUDITORIA TÉCNICA COMPLETA — 450 linhas de análise exaustiva
bda202c fix: CRITICAL — ban immigration terms from SaaS/Met/Dec documents
02db8da feat: PPTX Engineering Spec incorporado no sistema
```

---

18. PRÓXIMOS PASSOS RECOMENDADOS

Prioridade Alta

1. **Integrar USCIS Reviewer no pipeline** — O agente existe mas não é chamado automaticamente. Deveria ser Fase 1.6 (após Quality Gate, antes de SOC).

2. **Integrar Auto-Debugger com Quality Gate** — Quando o quality gate reprovar um documento, o auto-debugger deveria analisar as violações e propor regras automaticamente, fechando o loop de auto-learning.

3. **Reduzir taxa de falha (43%)** — Investigar as 17 gerações que falharam, categorizar as causas raiz, e criar fallbacks inteligentes.

4. **Geração paralela de cartas satélite** — Usar Agent Teams do Claude para gerar todas

as cartas simultaneamente, com variação estrutural (anti-ATLAS) garantida.

5. **Compliance USCIS Evidence Classifier** — O USCIS usa IA para categorizar evidências. Criar módulo que verifica se cada evidência está corretamente categorizada conforme o sistema da USCIS.

Prioridade Média

6. **Importação automática de perfil** — Integrar Google Scholar, LinkedIn, ResearchGate para enriquecer dados do cliente automaticamente.

7. **Scoring quantitativo de perfil** — Criar Case Predictor v2 com benchmarking quantitativo por critério (EB-1A) ou prong (EB-2 NIW).

8. **Atualizar docs_folder_path de todos os clientes** — Muitas pastas foram renomeadas (OMNI → 3_OMNI, _PROEX vs 2_PROEX). Corrigir registros.

9. **Gravar resultados do SOC no generations.json** — Score e problemas encontrados pela revisão cruzada ficam apenas no arquivo REVIEW, não no banco.

Prioridade Baixa

10. **Deploy em Vercel** — Atualmente roda apenas em localhost. A geração via `claude -p` é local, mas o frontend poderia ser deployado.

11. **Dashboard de métricas avançado** — Visualizar tendências de qualidade ao longo do tempo, regras mais ativadas, clientes com mais problemas.

12. **Integração Canva MCP** — Para geração de infográficos diretamente no pipeline.

Documento compilado em 08 de abril de 2026
Fonte: Codebase petition-engine + 8 auditorias + 42 gerações + 93 error_rules